

Conceptos de Sistemas Operativos 2012

Administración de Memoria - I



- ✓ Versión: Abril 2011
- ✓ Palabras Claves: Procesos, Espacio de Direcciones, Memoria, Seguridad, Particiones, Fragmentación

Algunas diapositivas han sido extraídas de las ofrecidas para docentes desde el libro de Stallings (Sistemas Operativos) y el de Silberschatz (Operating Systems Concepts). También se incluyen diapositivas cedidas por Microsoft S.A.



Administración de Memoria

- ✓ División Lógica de la Memoria para alojar a múltiples procesos
- ✓ La Memoria debe ser asignada eficientemente para contener el mayor número de procesos como sea posible.



Requisitos

☑ Reubicación

- ✓ El programador no debe ocuparse de conocer donde será colocado el programa para ser ejecutado.
- ✓ Mientras un proceso se ejecuta, puede ser sacado y traído a la memoria (swap) y colocarse en diferentes lugares.
- ✓ Las referencias a la memoria se deben traducir según dirección actual del proceso.



Requisitos (cont).

☑ Protección

- ✓ Los procesos no deben ser capaces de referencias direcciones de memoria de otros procesos (salvo que tengan permiso)
- ✓ El chequeo se debe realizar durante la ejecución:
 - ♦ El SO no puede anticipar todas las referencias a memoria que un proceso puede realizar.



Requisitos (cont).

☑ Compartición

- ✓ Permitir que varios procesos accedan a la misma porción de memoria.
 - ♦ Ej: Rutinas comunes, librerías, espacios explícitamente compartidos, etc.
- ✓ Lleva a un mejor uso de la memoria, evitando copias incesarías de instrucciones

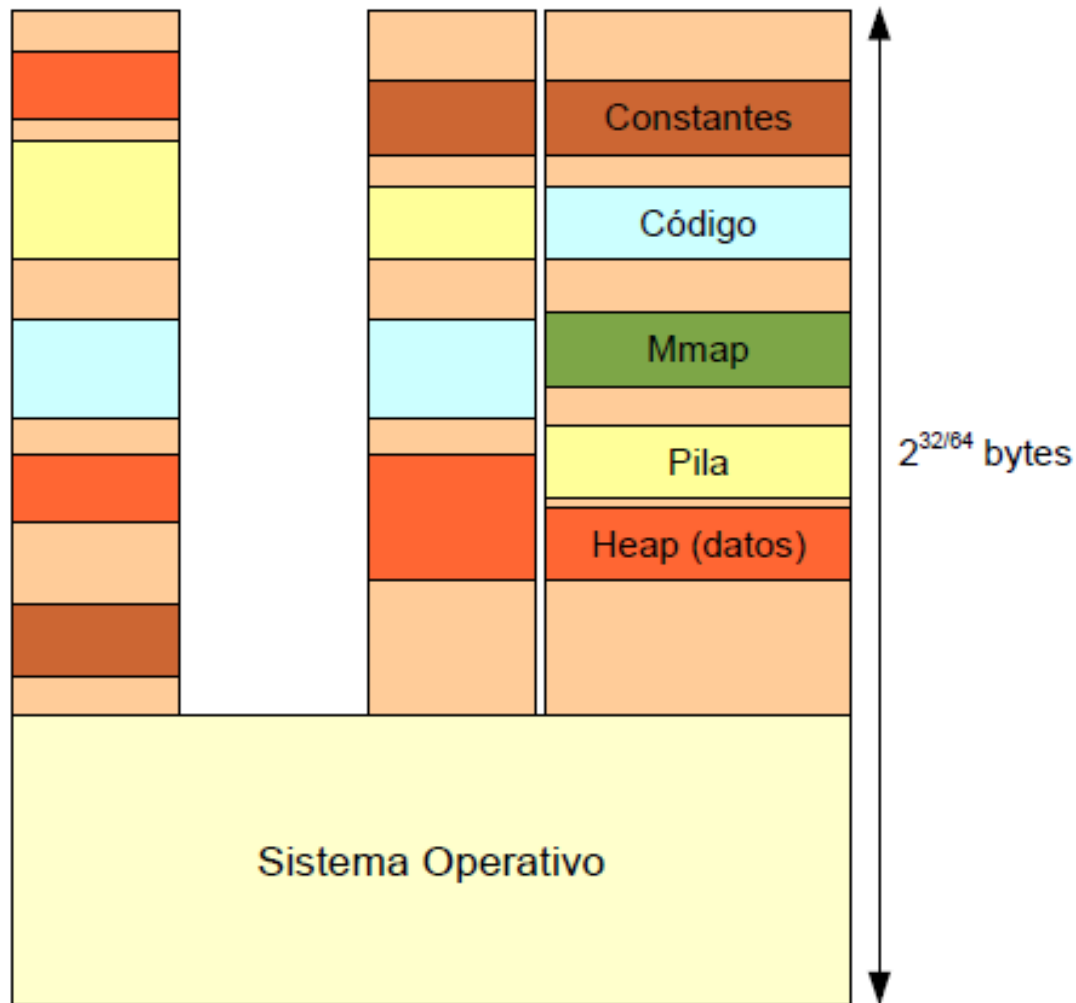


Abstracción - Espacio de Direcciones

- ✓ Rango de direcciones (a memoria) posibles que un proceso puede utilizar para direccionar sus instrucciones y datos.
- ✓ Varía en un rango dependiendo de la arquitectura
 - ✓ 32 bits: $0 \dots 2^{32} - 1$
 - ✓ 64 bits: $0 \dots 2^{64} - 1$
- ✓ Debe ser independiente de la ubicación “real” del proceso en la memoria



Abstracción -Espacio de Direcciones (cont)



Direcciones

☑ Lógicas

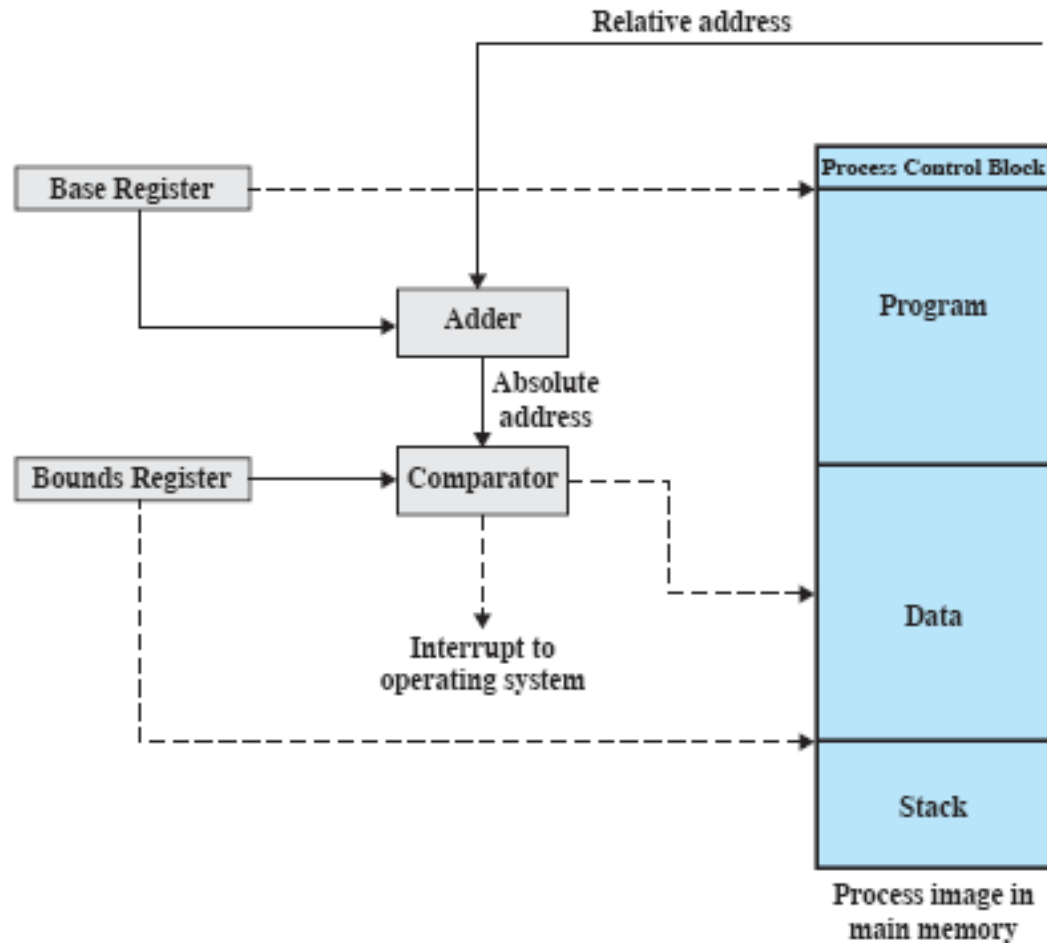
- ✓ Referencia a una localidad de memoria independiente de la asignación actual de los datos en la memoria.
- ✓ Se debe realizar una traducción a una dirección física.

☑ Físicas

- ✓ La dirección absoluta en la memoria principal.



Direcciones (cont.)



Direcciones - Registros utilizados

- ☑ Registro Base
 - ✓ Dirección de comienzo del proceso.
- ☑ Registro Limite
 - ✓ Dirección final del proceso o medida del proceso.
- ☑ Su valor se fija cuando el proceso es cargado a memoria (Context Switch)



Direcciones - Registros utilizados

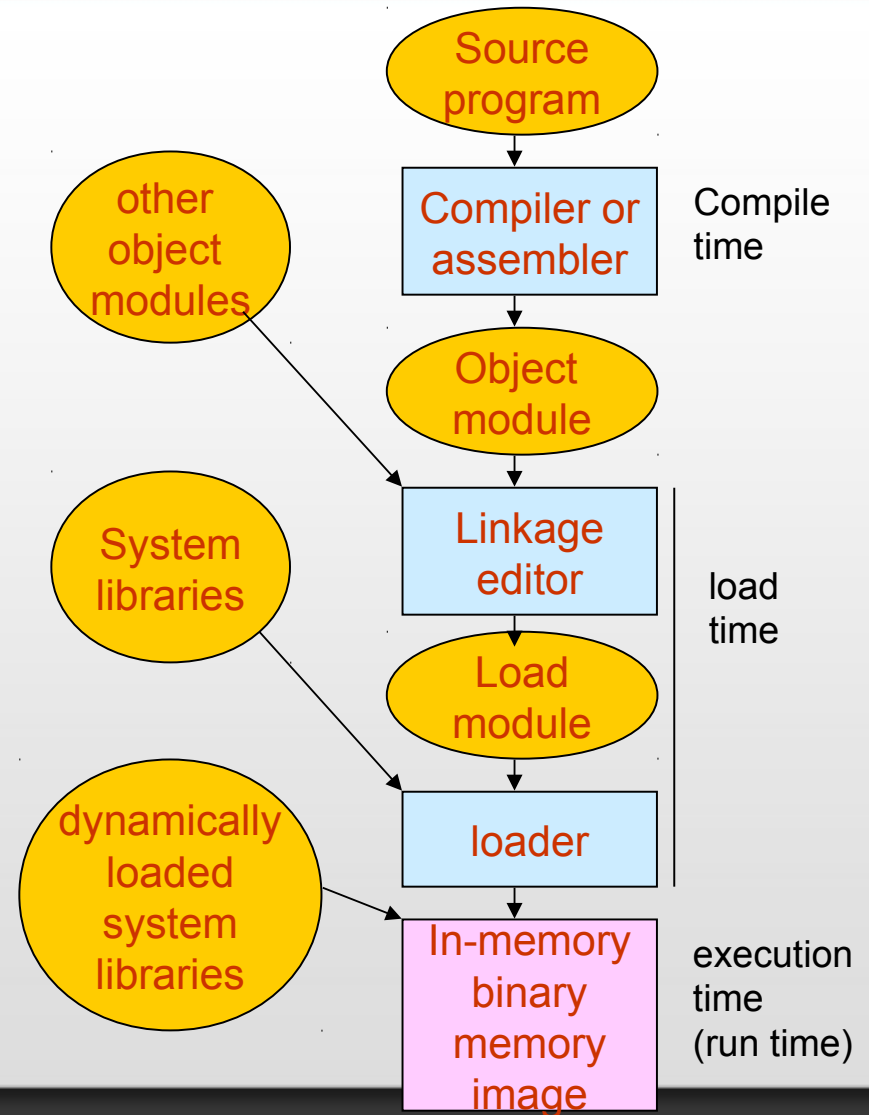
(cont)

- ☑ Se utiliza la dirección lógica junto con el registro base para obtener una dirección física (según la técnica utilizada).
- ☑ El resultado se compara con el valor del registro limite (según la técnica utilizada).
- ☑ Si la dirección generada es incorrecta, se genera una interrupción al SO.



Binding de direcciones

- ✓ Las direcciones en los programas fuentes son simbólicas
- ✓ Compiladores relaciones direcciones simbólicas a direcciones reubicables
- ✓ El linkeditor/cargador relaciona las direcciones reubicables en direcciones absolutas



Dir. Lógicas vs. Físicas

- ❑ La CPU trabaja con direcciones lógicas. Para acceder a memoria principal, se deben transformar en direcciones absolutas
- ❑ Resolución de direcciones (address-binding): transformar la dirección lógica en la dirección física correspondiente
- ❑ Resolución en momento de compilación y en tiempo de carga
 - ✓ Direcciones Lógicas y Físicas son idénticas
- ❑ Resolución en tiempo de ejecución
 - ✓ Direcciones Lógicas y Físicas son diferentes
 - ✓ Direcciones Lógicas son llamadas “Direcciones Virtuales”
 - ✓ El mapeo entre “Virtuales” y “Físicas” es realizado por hardware (MMU (Memory Management Unit))

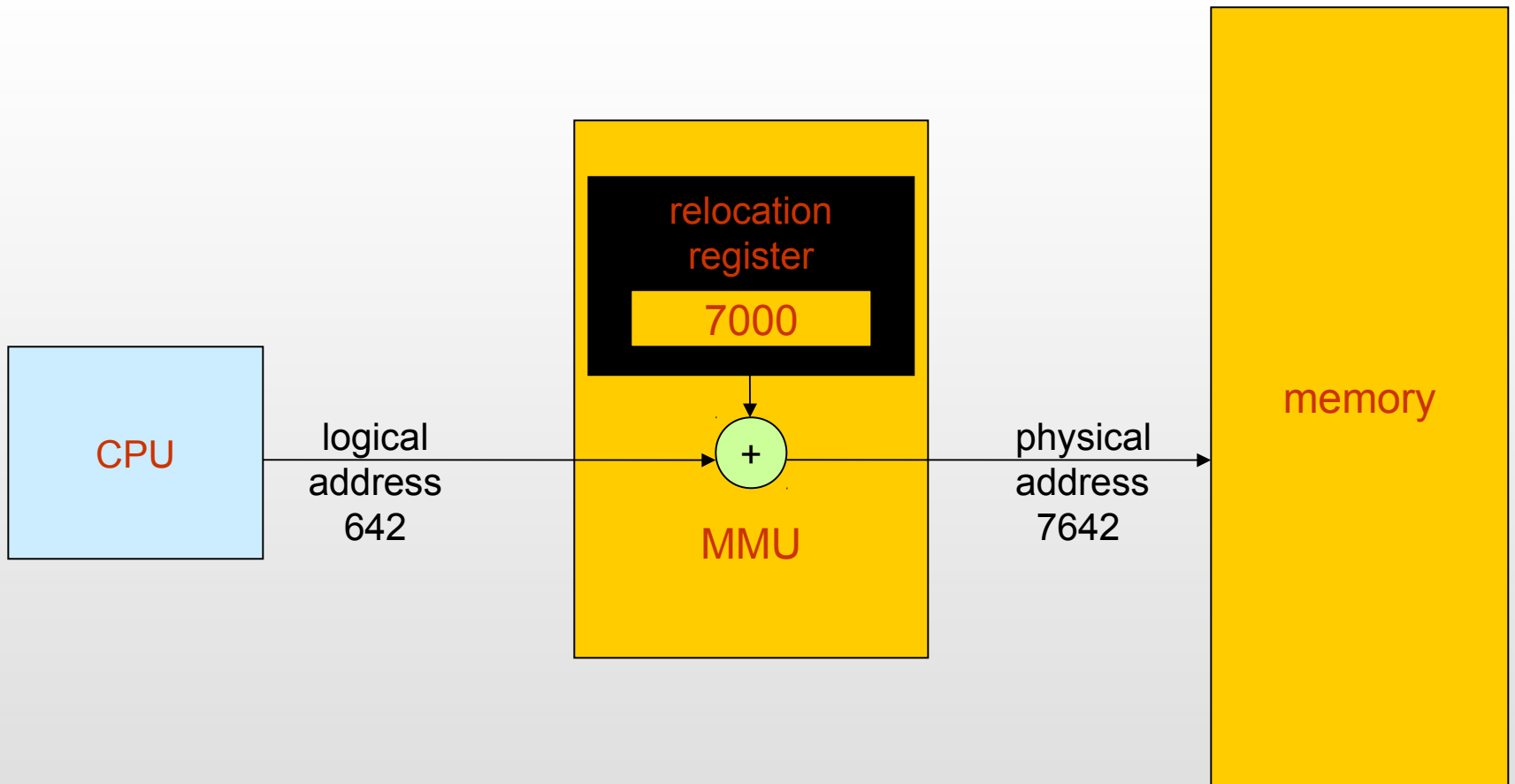


Memory Management Unit (MMU)

- ☑ Dispositivo de Hardware que mapea direcciones virtuales a físicas
 - ✓ Es parte del Procesador
 - ✓ Re-programar el MMU es una operación privilegiada que solo puede ser realizada en Kernel Mode
- ☑ El valor en el “registro de realocación” es sumado a cada dirección generada por el proceso de usuario al momento de acceder a la memoria.
 - ✓ Los procesos nunca ven direcciones físicas



MMU



Carga Dinámica

- ☑ Una rutina no es cargada hasta que no es llamada
- ☑ Todas las rutinas se mantienen en el disco en un formato “reubicable”
- ☑ Cuando una rutina invoca a otra:
 - ✓ Se chequea que ya no haya sido cargada
 - ✓ Si no se cargo, se llama al “cargador” para que la cargue
 - ✓ El “cargador” actualiza el espacio de direcciones del proceso
 - ✓ El control es pasado a la nueva rutina
- ☑ Mejor utilización de la memoria
 - ✓ Rutinas no utilizadas no son nunca cargadas



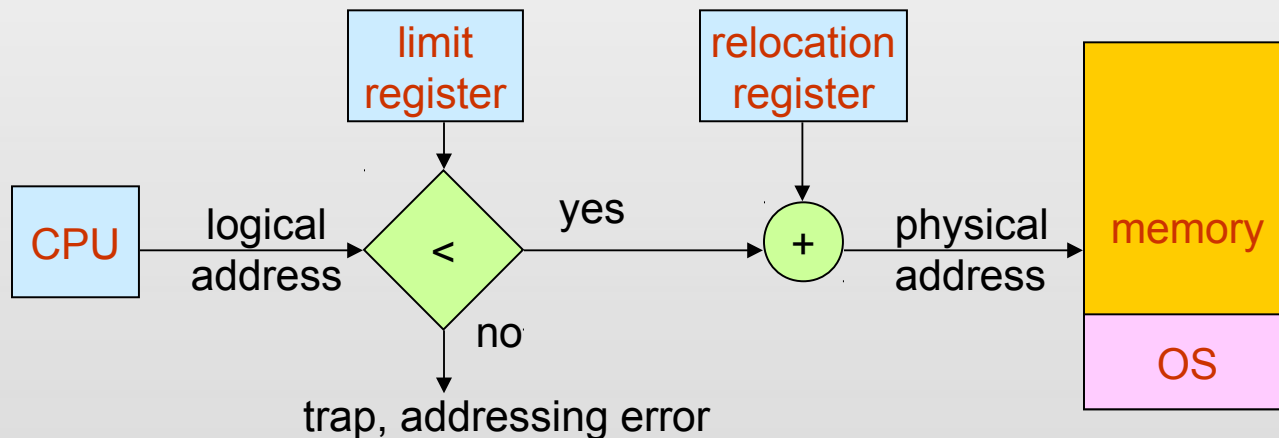
Linkeo Dinámico (*Dynamic linking*)

- ☑ Similar a la carga dinámica
 - ✓ El linkeo es postergado hasta el momento de “run-time”
 - ✓ Las librerías dinámicas no son “adjuntadas estáticamente” al proceso, solo un pequeño “stub” es adjuntado.
 - ✓ El “stub” indica como cargar la rutina apropiada
- ☑ Todos los programas pueden utilizar la misma copia de una librería (código) (DLLs)
- ☑ El Linkeo Dinámico requiere soporte del S.O.
 - ✓ S.O. es el único que puede localizar una librería en el espacio de direcciones de otro proceso



Asignación de Memoria

- ✓ La memoria principal debe ser organizada para contener el S.O. (rutinas, librerías, estructuras) y procesos
 - ✓ El S.O. debe protegerse de cambio de los procesos
 - ✓ Procesos deben protegerse unos de otros
- ✓ Única Partición:
 - ✓ Los procesos ocupan una única partición de memoria
 - ✓ La protección se implementa por un “limite” y un registro de “reubicación”



Particiones

- ✓ La memoria es dividida en varias regiones (particiones).
- ✓ Los procesos (su espacio de direcciones) se colocan en las particiones según su tamaño.
- ✓ Técnicas:
 - ✓ Particiones Fijas
 - ✓ Particiones Dinámicas



Particiones Fijas

- ✓ Regiones definidas con limites fijados
- ✓ Tamaños:
 - ✓ Igual Tamaño
 - ♦ Cualquier proceso cuyo tamaño es menor igual que la partición puede ser colocado en una partición libre.
 - ♦ Si todas están ocupadas → Swap
 - ♦ Un programa puede no entrar en la partición
 - ♦ Ineficiencia: Cualquier programa, sin importar su tamaño, ocupara una partición



Particiones Fijas (cont).

✓ Diferente Tamaño

- ♦ Evita el problema de las particiones de igual tamaño.
- ♦ Complejidad en el algoritmo de selección de partición para un proceso.



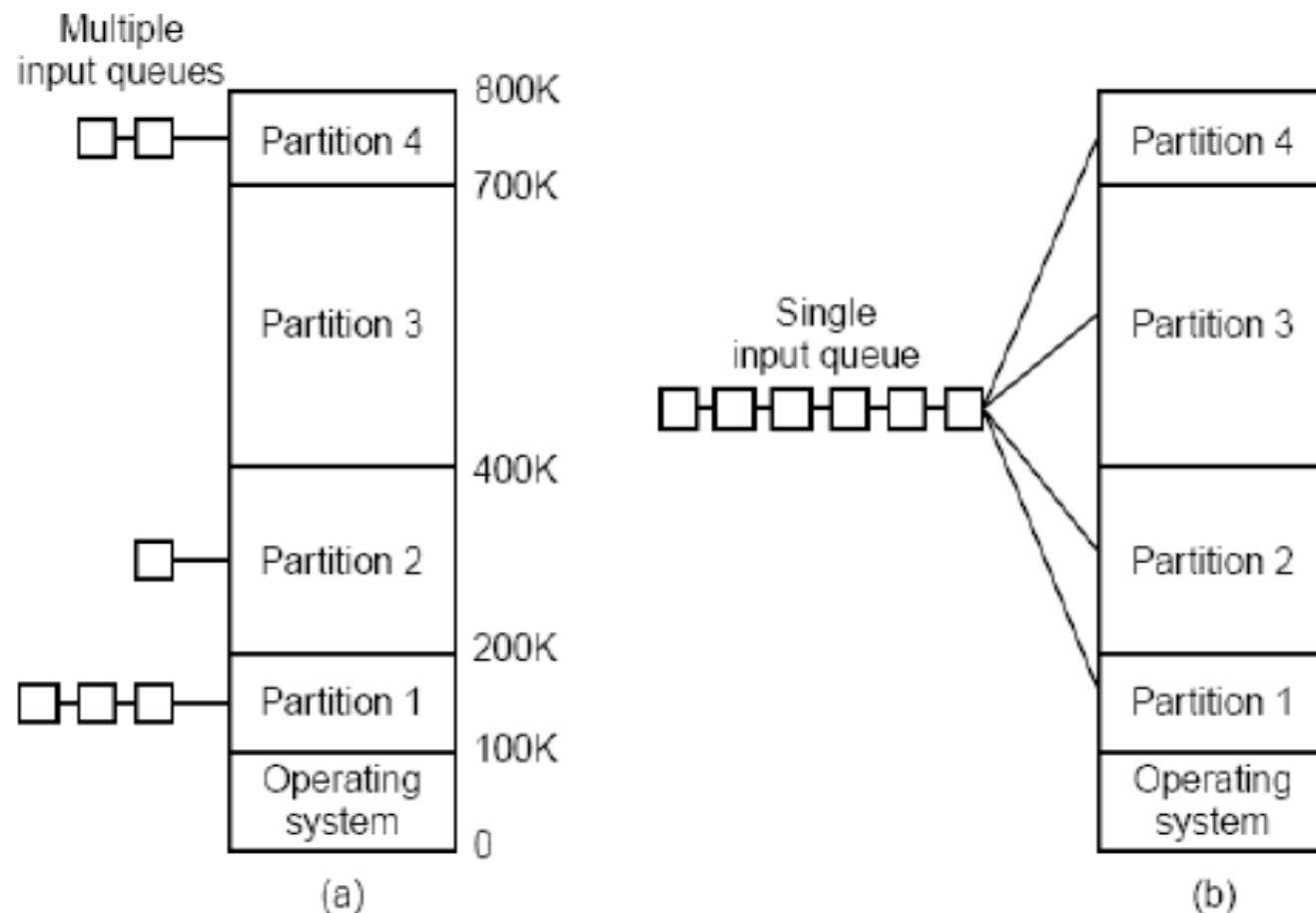
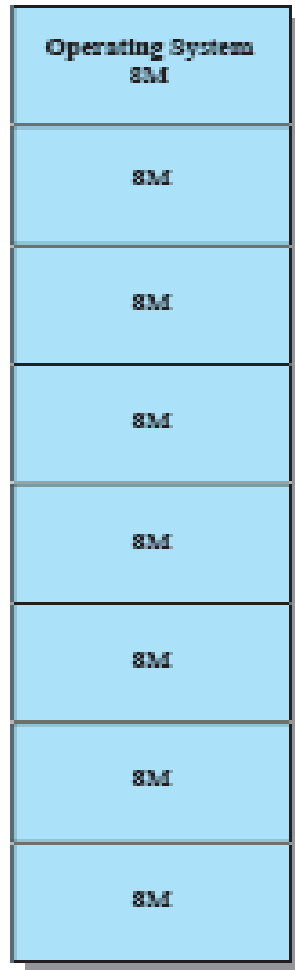
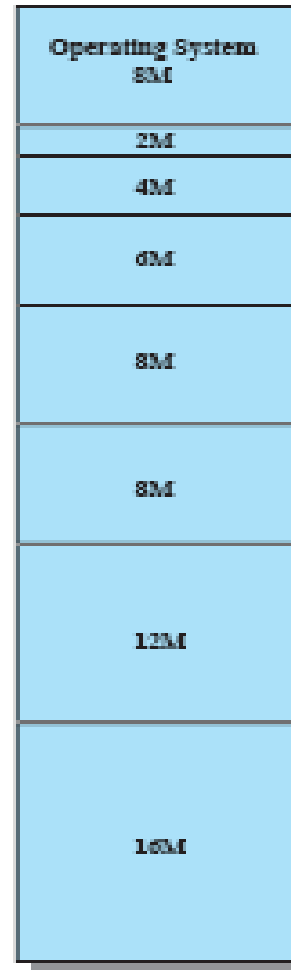


Fig. 4-2. (a) Fixed memory partitions with separate input queues for each partition. (b) Fixed memory partitions with a single input queue.

Particiones Fijas (cont.)



(a) Equal-size partitions



(b) Unequal-size partitions

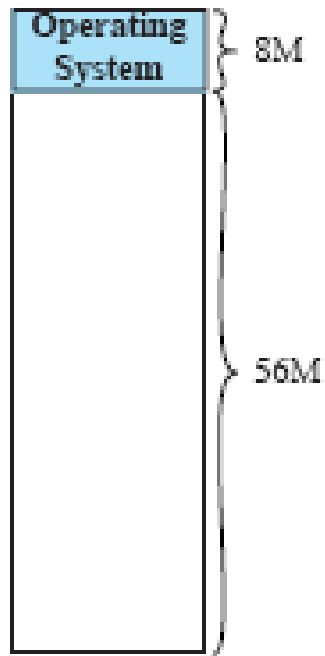


Particiones Dinámicas

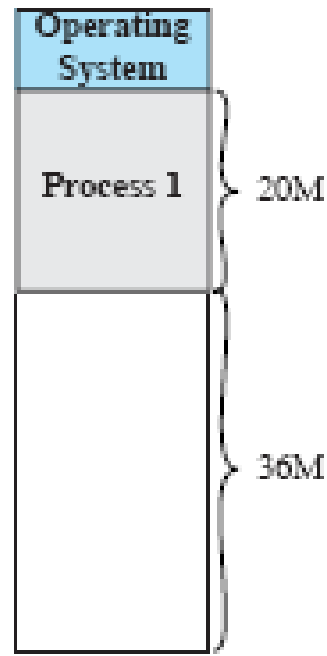
- ✓ Las particiones son de tamaño y número variable
- ✓ Los procesos son colocados exactamente en particiones de igual a su tamaño (generadas dinámicamente)



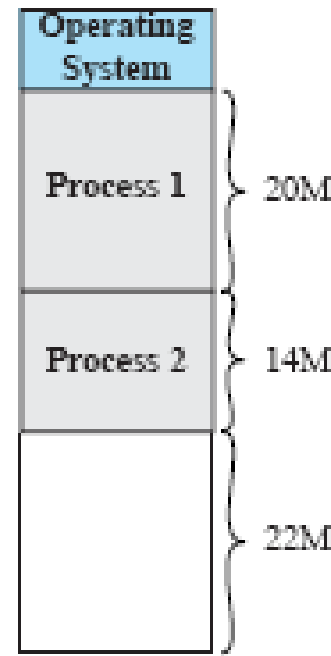
Particiones Dinámicas (cont.)



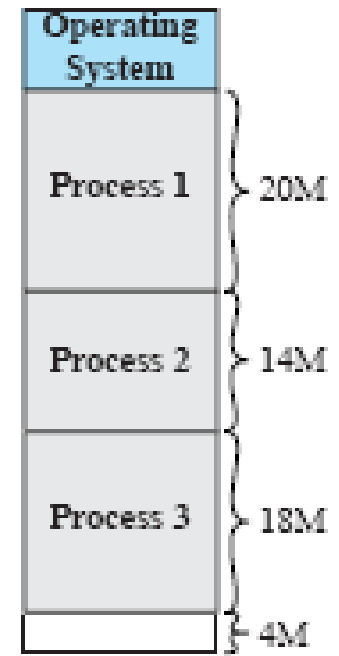
(a)



(b)



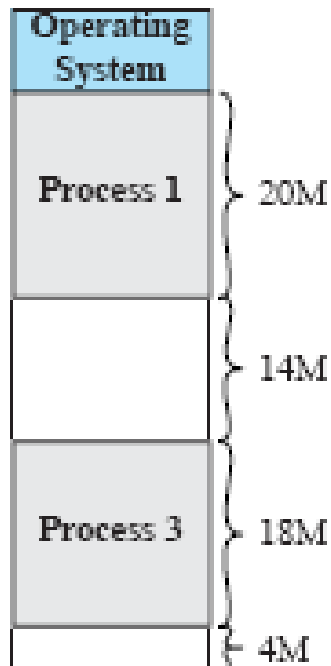
(c)



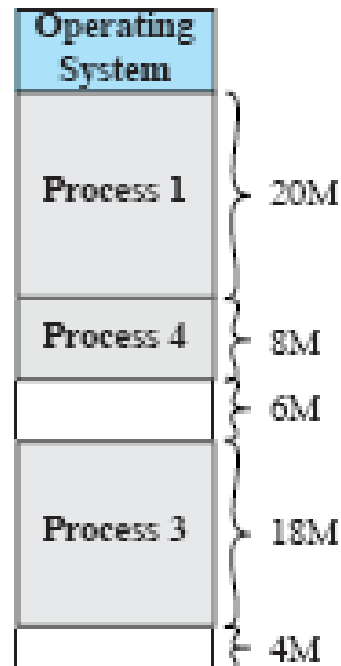
(d)



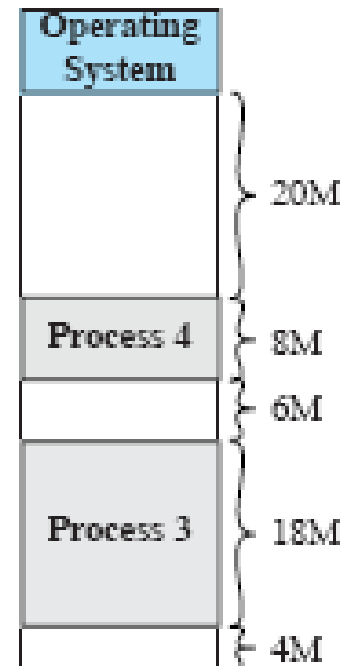
Particiones Dinámicas (cont.)



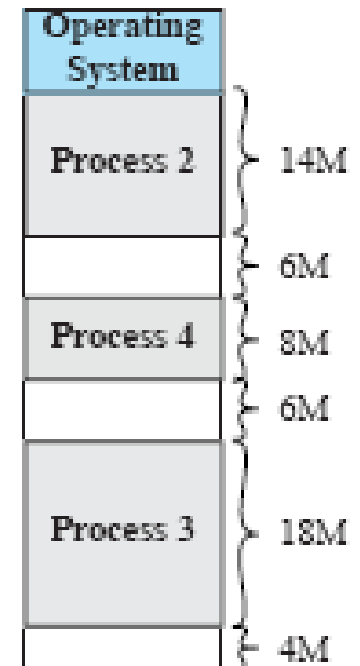
(e)



(f)



(g)



(h)



Fragmentación

- ☑ Espacio libre de la memoria que no puede ser utilizado por NO encontrarse de manera contigua
- ☑ Interna → Particiones Fijas
 - ✓ Espacio dentro de la partición sin utilizar
- ☑ Externa → Particiones Dinámicas
 - ✓ Cada vez que entra y sale un proceso se genera huecos en la memoria, en los que eventualmente un proceso no podría entrar, pero si entraría si unimos todos los huecos → COMPACTACION



Particiones - Algoritmos de Ubicación

- ☑ Particiones Fijas de igual tamaño
 - ✓ No se necesita un algoritmo
- ☑ Particiones Fijas de diferente tamaño y Particiones Dinámicas:
 - ✓ First Fit
 - ✓ Best Fit
 - ✓ Worst Fit
 - ✓ Next Fit



Particiones - Algoritmos de Ubicación (cont.)

☑ Best fit

- ✓ Selecciona la partición mas pequeña que contiene al proceso
- ✓ Mucho overhead en la búsqueda
- ✓ En particiones dinámicas: se generan muchos huecos pequeños de memoria libre → Fragmentación externa

☑ First Fit

- ✓ Recorre las particiones libres en orden, buscando la primera que contenga el proceso.



Particiones - Algoritmos de Ubicación (cont.)

☑ Next fit

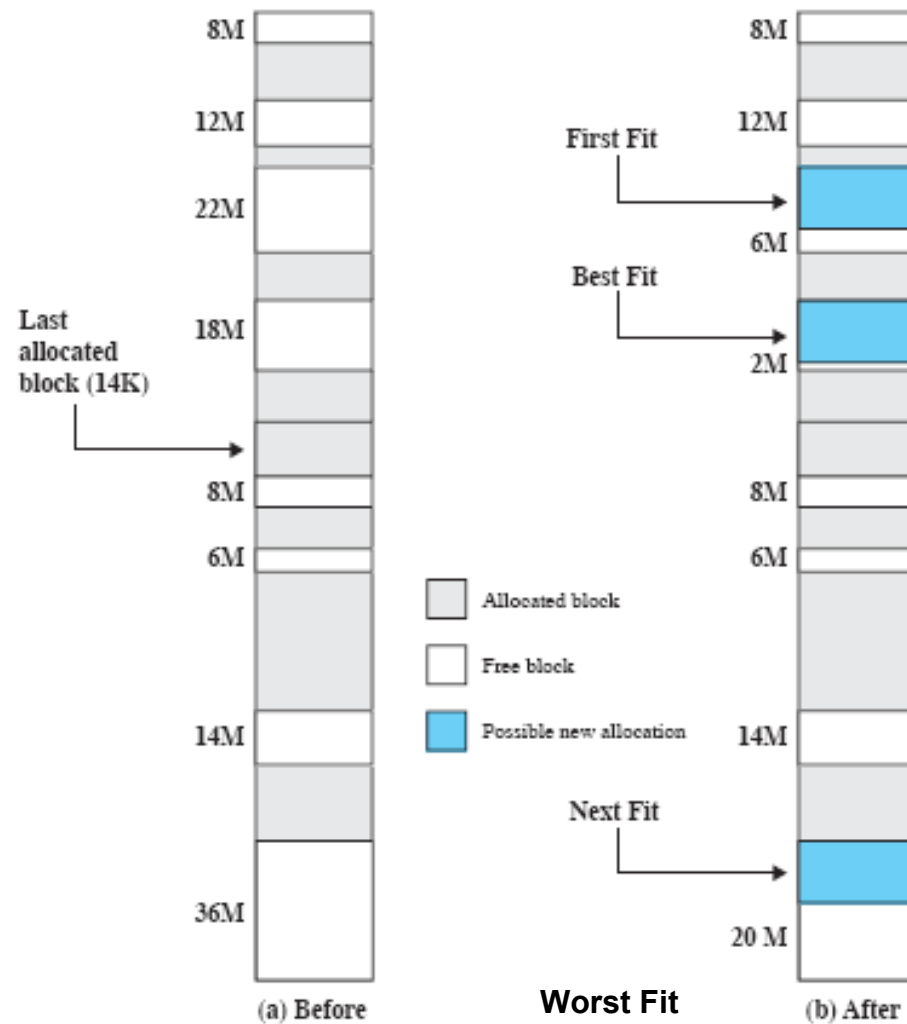
- ✓ Mantiene las particiones libres como una lista circular.
- ✓ Selecciona la primer partición que encuentra que contenga el proceso

☑ Worst Fit

- ✓ Selecciona la partición libre mas grande que contenga el proceso.
- ✓ Mal uso en Particiones Fijas
- ✓ Buen uso en Particiones Dinámicas.



Particiones - Algoritmos de Ubicación (cont.)



Particiones - Manejo de los "huecos"

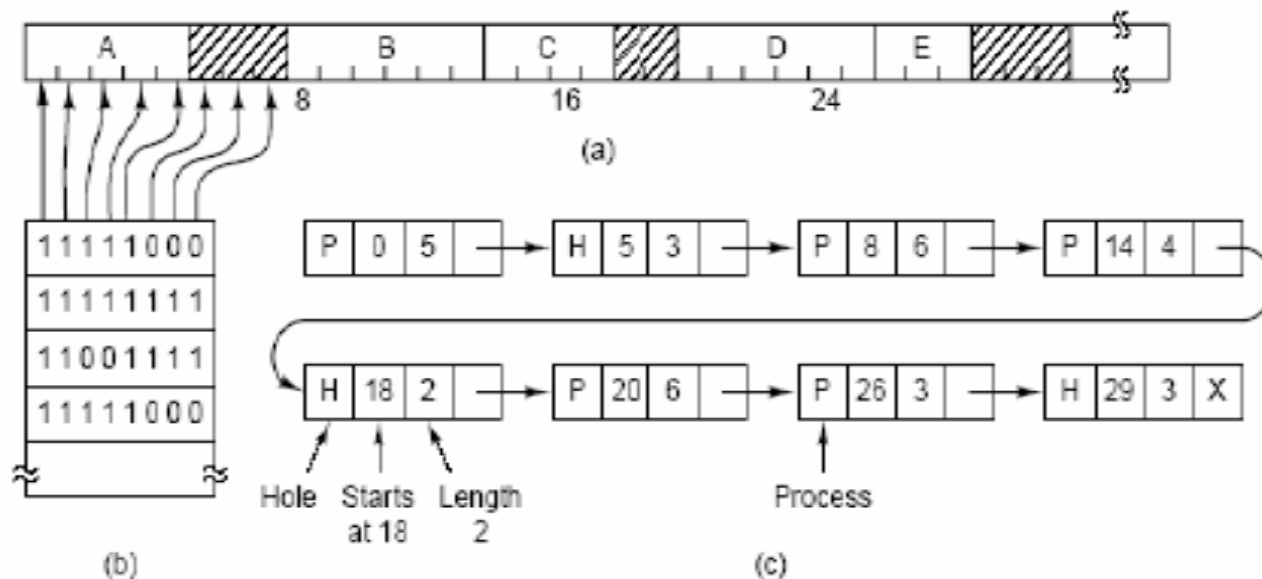


Fig. 4-7. (a) A part of memory with five processes and three holes. The tick marks show the memory allocation units. The shaded regions (0 in the bitmap) are free. (b) The corresponding bitmap. (c) The same information as a list.



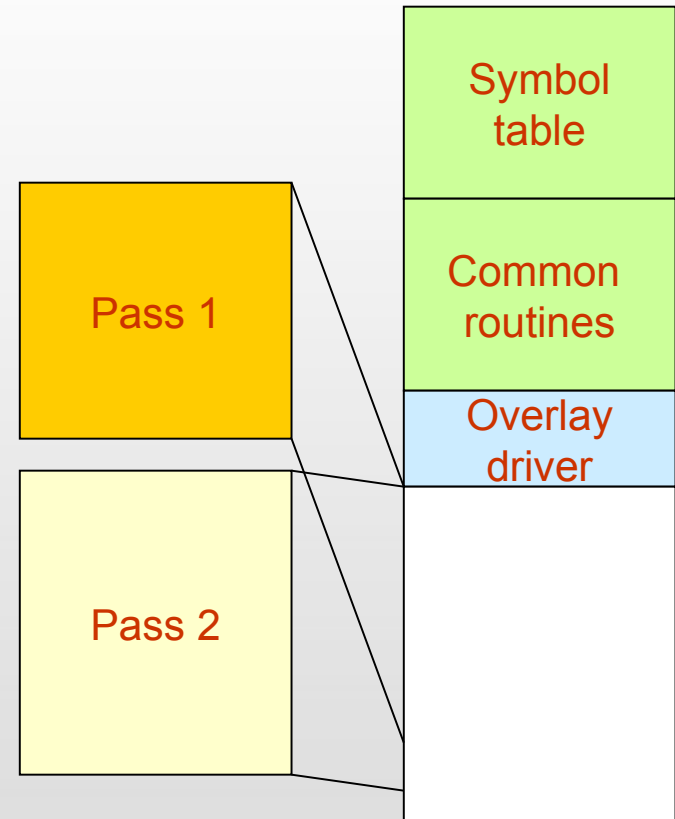
Superposiciones (overlapping)

- ✓ El tamaño de los procesos y sus datos puede exceder el tamaño de la memoria

Conceptos:

- ✓ Separar el programa en módulos
- ✓ Cargar los módulos alternativamente
- ✓ El dispositivo de “superposiciones” localiza los módulos en el disco
- ✓ Los módulos superpuestos se guardan como imágenes absolutas de memoria
- ✓ Se requiere soporte de los compiladores

Example:
multi-pass compiler



Swapping

- ✓ Un proceso puede ser temporalmente sacados de la memoria (**swapped out**) a un disco de manera de permitir la ejecución de procesos.

Si se descarga considerando las direcciones físicas

- ✓ Al hacer **swapped in** se debe cargar en el mismo espacio de memoria que ocupaba antes

Si se descarga considerando las direcciones lógicas

- ✓ Al hacer **swapped in** se puede cargar en cualquier espacio de direcciones de memoria

